Module 07 Lab – Better Model Evaluation

Objective: Move beyond simple accuracy and learn to evaluate a model using a confusion matrix, precision, recall, F1-score, and cross-validation.

Part 1: Setup and Model Training

Train a logistic regression model using the Titanic dataset to predict survival.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Load and prepare the data.
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/da
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
df['Age'] = df['Age'].fillna(df['Age'].median())

features = ['Age', 'Pclass', 'Sex', 'Fare']
target = 'Survived'
X = df[features]
y = df[target]

# Split data into training and testing sets.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train the model and make predictions.
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print('Training rows:', len(X_train))
print('Testing rows:', len(X_test))
print('Model training complete.')
```

```
Training rows: 712
Testing rows: 179
Model training complete.
```

Part 2: The Confusion Matrix

A confusion matrix shows which predictions were correct and which were incorrect.

- **True positive:** Predicted survival, and the passenger survived.
- **True negative:** Predicted did not survive, and the passenger did not survive.
- **False positive:** Predicted survival, but the passenger did not survive.
- **False negative:** Predicted did not survive, but the passenger survived.

Task 1: Generate and Plot a Confusion Matrix

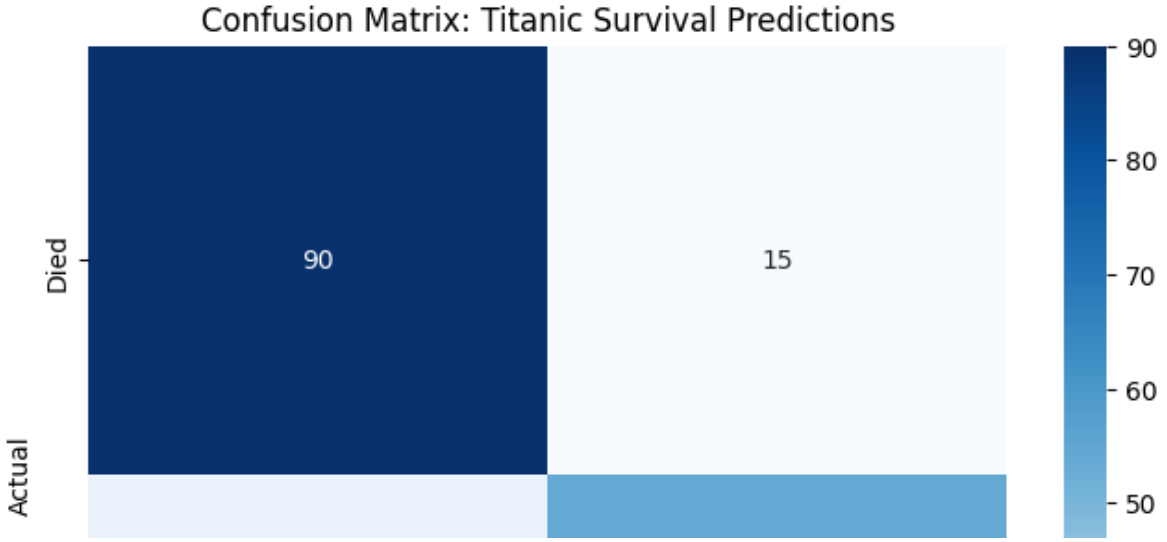
Use `confusion_matrix` and `seaborn.heatmap` to display the results.

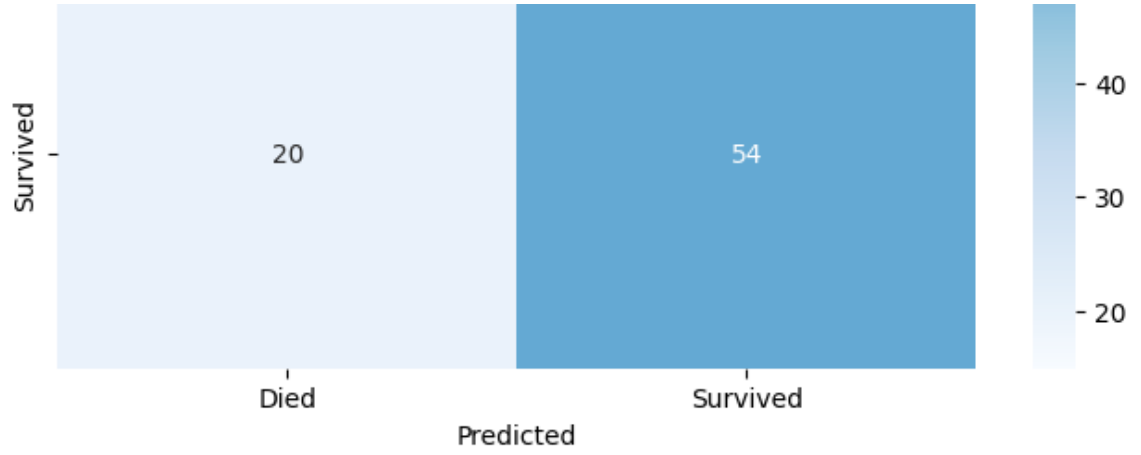
```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Calculate the confusion matrix.
cm = confusion_matrix(y_test, y_pred)
print('Confusion matrix:')
print(cm)

# Plot the confusion matrix.
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Died', 'Survived'],
            yticklabels=['Died', 'Survived'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix: Titanic Survival Predictions')
plt.show()
```

```
Confusion matrix:
[[90 15]
 [20 54]]
```





Part 3: Precision, Recall, and F1-Score

- **Precision:** Of the predicted survivors, how many survived?
- **Recall:** Of the actual survivors, how many were correctly predicted?
- **F1-score:** A score that balances precision and recall.

Task 2: Generate a Classification Report

Use `classification_report` to show precision, recall, and F1-score for both classes.

```
from sklearn.metrics import classification_report

# Print precision, recall, and F1-score for each class.
report = classification_report(y_test, y_pred, target_names=['Died',
print(report)
```

```
              precision    recall  f1-score   support

        Died       0.82      0.86      0.84       105
    Survived       0.78      0.73      0.76        74

    accuracy                           0.80       179
   macro avg       0.80      0.79      0.80       179
weighted avg       0.80      0.80      0.80       179
```

Part 4: Cross-Validation

A single train-test split might not represent every part of the data. Five-fold cross-validation tests the model five times using different parts of the data.

Task 3: Perform Cross-Validation

Use `cross_val_score` with five folds. Display all five scores, the average, and the standard deviation.

```
from sklearn.model_selection import cross_val_score

# Use five-fold cross-validation on the full dataset.
cv_scores = cross_val_score(model, X, y, cv=5)
print(f'Scores for each fold: {cv_scores}')
print(f'Average CV Score: {cv_scores.mean():.2%}')
print(f'Standard Deviation of CV Scores: {cv_scores.std():.4f}')
```

```
Scores for each fold: [0.7877095  0.78089888 0.78651685 0.7752809  0.8
Average CV Score: 78.68%
Standard Deviation of CV Scores: 0.0094
```

Knowledge Check

1. Describe a real-world scenario where precision matters more than recall.

A spam filter is one example. I would want a message marked as spam to actually be spam, because an important email could otherwise be sent to the spam folder.

2. Describe a real-world scenario where recall matters more than precision.

A screening test for a serious illness is one example. It is important to identify as many people who may have the illness as possible, even if some people need additional testing to confirm the result.

3. Why is a cross-validation score generally more trustworthy than a score from one train-test split?

Cross-validation tests the model on several different parts of the dataset instead of relying on one split. The average of those scores gives a better idea of how consistently the model performs.